

Artifacts of the user study described in our paper, extracted from:
[22] V. C. Honorato, AI-Assisted formal specification and verification in Dafny: A multi-provider VS code extension, Master's thesis, Faculty of Engineering, University of Porto, available at <https://hdl.handle.net/10216/169102> (2025).

Appendix B

Artifacts of Experimental Trial

This appendix provides comprehensive instructions for the installation, configuration, and operation of the enhanced Dafny VSCode extension. This extension provides complete IDE integration for Dafny with AI-powered code generation capabilities, including loop invariant generation, precondition and postcondition synthesis, supporting OpenAI, Claude, and DeepSeek API integration for automated formal verification assistance.

B.1 Installation Procedures

B.1.1 Removal of Previous Dafny Extensions

Important: If you have an existing Dafny extension installed, it must be removed first:

1. **Open Extensions View**

- Use `Ctrl+Shift+X` (Windows/Linux) or `Cmd+Shift+X` (macOS)

2. **Locate Existing Dafny Extension**

- Search for "Dafny" in the extensions list
- Identify any previously installed Dafny extensions

3. **Uninstall Previous Extension**

- Click the gear icon next to the existing extension
- Select "Uninstall" from the dropdown menu

B.1.2 Installing the Enhanced Dafny Extension

1. **Download the VSIX Package**

- Navigate to the [releases page](https://github.com/emantrigo/dafny-plugin/releases)¹

¹<https://github.com/emantrigo/dafny-plugin/releases>

- Download the latest `.vsix` file from the repository

2. Install from VSIX File

- In Visual Studio Code, access the Extensions view
- Click the "... " (More Actions) menu at the top of the Extensions view
- Select "Install from VSIX..."
- Navigate to the downloaded `.vsix` file and select it
- Confirm the installation

3. Complete Installation

- Restart Visual Studio Code to ensure proper extension loading
- The Dafny Language Server will be installed automatically upon first use

B.2 Configuration Settings

B.2.1 Accessing Extension Settings

1. Open Settings Interface

- Navigate to File > Preferences > Settings (Windows/Linux)
- Or use `Cmd+,` on macOS
- Alternatively, use the keyboard shortcut `Ctrl+,`

2. Locate Dafny Settings

- Search for "Dafny" in the settings search bar
- This will filter and display all Dafny-related configuration options

B.2.2 AI Model Selection Interface

The extension incorporates an intuitive model selection interface integrated into the VSCode status bar for streamlined AI provider management:

1. Status Bar Integration

- The model selector is positioned in the VSCode status bar (lower-right corner)
- Displays "Auto" when operating in automatic mode
- Shows the provider name when operating in manual selection mode

2. Automatic Mode (Default)

- The extension autonomously selects the optimal available provider

- Implements intelligent fallback mechanisms across configured providers
- Maximizes generation success rates through provider redundancy

3. Manual Selection Mode

- Click the status bar item to access provider selection interface
- Choose from available providers: OpenAI, Claude, or DeepSeek
- Selected provider receives priority in the generation attempt sequence

B.2.3 API Configuration

The extension requires API key configuration for at least one supported provider:

B.2.3.1 OpenAI Configuration

- Setting: "Dafny: Open AI API Key"
- Input: Valid OpenAI API key with appropriate access permissions

B.2.3.2 Claude Configuration

- Setting: "Dafny: Claude API Key"
- Input: Valid Anthropic Claude API key

B.2.3.3 DeepSeek Configuration

- Setting: "Dafny: DeepSeek API Key"
- Input: Valid DeepSeek API key

B.2.4 Advanced Configuration Options

Retry Configuration:

- **"Dafny: Number Of Retries"**: Set the number of attempts the AI makes to generate valid loop invariants
- Default value can be adjusted based on complexity requirements and API limitations

B.3 Operation and Usage

B.3.1 Basic Dafny Development Features

Real-time Verification:

- Verification occurs automatically as you type

- Syntax errors and verification issues are highlighted in real-time

Navigation Features:

- Go-to-definition: Navigate to symbol definitions
- Hover information: View detailed information about symbols

B.3.2 AI-Powered Loop Invariant Generation**B.3.2.1 Generation Process****1. Code Selection**

- Open the target Dafny file (.dfy extension)
- Select the portion of code containing the loop requiring invariants
- Ensure the selection encompasses the complete loop structure

2. Invoke Generation

- **Method 1:** Right-click to open the Dafny context menu and select "Generate Loop Invariants"
- **Method 2:** Use the keyboard shortcut `Ctrl+F9`

3. AI Processing

- The extension communicates with the configured AI provider (OpenAI, Claude, or DeepSeek)
- Multiple attempts may be made based on the retry configuration
- Generated invariants are automatically validated by the Dafny verifier

B.3.2.2 AI Provider Selection Strategy

The extension provides two operational modes for the utilization of AI providers:

1. Automatic Provider Selection

- The system maintains the status bar indicator in "Auto" mode
- Sequential provider attempts are made until successful code generation
- Optimizes success probability through intelligent provider ordering

2. Manual Provider Selection

- Users may specify a preferred provider through the status bar interface
- Selected provider receives initial generation attempt priority
- Fallback to alternative providers occurs upon generation failure

B.3.2.3 Precondition and Postcondition Generation

The extension also supports AI-powered generation of method specifications:

1. Method Selection

- Select the method or function requiring specifications
- Ensure the entire method signature and comments are included in the selection

2. Specification Generation

- Right-click to access the context menu
- Select "Generate Preconditions" or "Generate Postconditions" as needed
- The AI analyzes the method logic to suggest appropriate specifications

3. Automated Verification

- Generated preconditions (`requires` clauses) define input constraints
- Generated postconditions (`ensures` clauses) specify output guarantees
- All specifications are verified for logical consistency with the method implementation

B.3.2.4 Result Handling

- **Successful Generation:** Valid specifications are integrated into the code
- **Failed Generation:** Error messages indicate unsuccessful attempts with suggestions for manual refinement
- **Verification:** All generated specifications (invariants, preconditions, postconditions) are automatically verified for correctness
- **Iterative Refinement:** Users can modify generated specifications and re-verify as needed.

B.4 Troubleshooting

B.4.1 Common Installation Issues

Extension Conflicts:

- Ensure all previous Dafny extensions are completely uninstalled
- Restart Visual Studio Code after uninstallation

B.4.2 AI Generation Issues

Model Selection Issues:

- Verify that at least one API key is properly configured and valid
- Utilize automatic mode for optimal reliability and success rates
- Ensure API keys possess sufficient quota and appropriate permissions
- Network connectivity issues may affect provider availability

API Key Problems:

- Verify the correct API key is entered for the selected provider
- Ensure API key has sufficient credits/permissions
- Test API connectivity outside the extension if issues persist

Generation Failures:

- Review code structure for logical completeness
- Ensure loops are properly formed and syntactically correct
- Consider increasing the retry count for complex code structures

B.4.3 Performance Considerations

Verification Speed:

- Large files may require additional processing time
- Complex verification conditions increase processing duration
- Network latency affects AI API response times

B.5 Advanced Features

B.5.1 Counterexample Analysis

When verification fails, the extension provides detailed counterexample information:

- Visual highlighting of problematic code sections
- Detailed explanations of verification failures
- Interactive exploration of counterexample scenarios

B.6 Support and Development

This extension represents an advanced integration of formal verification tools with modern AI capabilities. For technical support, bug reports, or contributions, please refer to the project repository at <https://github.com/emantrigo/dafny-plugin>.

B.7 Experimental Exercises

This section presents the formal verification exercises used in the experimental evaluation, presented in the order they were administered during the experimental trial.

B.7.1 Exercise A1: Catalan Numbers

Listing B.1: Catalan number computation with loop invariant

```

1  /*
2   Catalan numbers can be defined recursively as  $C(0) = 1$  and  $C(n) = C(n-1) * (4n - 2) / (n + 1)$ .
3   Source: https://en.wikipedia.org/wiki/Catalan\_number
4  */
5
6  // Recursive definition.
7  ghost function C(n: nat): nat {
8    if n == 0 then 1 else C(n-1) * (4 * n - 2) / (n + 1)
9  }
10
11 // Iterative calculation of the n-th Catalan number in time  $O(n)$  and space  $O(1)$ .
12 method Catalan(n: nat) returns (res : nat)
13 {
14   res := 1; // C(0)
15   for i := 1 to n + 1
16   {
17     res := res * (4 * i - 2) / (i + 1);
18   }
19   return res;
20 }
21
22 // Test cases checked statically.
23 method TestCatalan() {
24   var r0 := Catalan(0); assert r0 == 1;
25   var r1 := Catalan(1); assert r1 == 1;
26   var r2 := Catalan(2); assert r2 == 2;
27   var r3 := Catalan(3); assert r3 == 5;
28 }
```

B.7.2 Exercise A2: Prime Number Detection

Listing B.2: Prime number verification with loop invariant

```
1 // Checks if a natural number is prime, i.e., it is greater than 1
2 // and not divisible by any number other than 1 and itself.
3 method IsPrime(n: nat) returns (result: bool)
4 {
5     if n <= 1 {
6         return false;
7     }
8     for i := 2 to n / 2 + 1
9     {
10         if n % i == 0 {
11             return false;
12         }
13     }
14     return true;
15 }
16
17 // Test cases checked statically by Dafny.
18 method TestIsPrime()
19 {
20     // prime number
21     var res1 := IsPrime(2);
22     assert res1;
23
24     // non-prime number
25     var res2 := IsPrime(8);
26     assert 8 % 2 == 0; // proof helper (counter-example)
27     assert ! res2;
28 }
```

B.7.3 Exercise B1: Recursive Minimum Finding

Listing B.3: Recursive minimum finding with specifications

```
1 // Find the smallest value (minimum) in the first 'n' positions (n > 0) of an array 'a'.
2 method FindMin(a: array<int>, n: nat := a.Length) returns (min: int)
3 {
4   if n == 1 {
5     return a[0];
6   }
7   else {
8     var m := FindMin(a, n - 1);
9     return if a[n - 1] < m then a[n - 1] else m;
10  }
11 }
12
13 // Test case checked statically.
14 method TestFindMin() {
15   var a : array<int> := new int[] [3, 2, 1, 4];
16   var min := FindMin(a);
17   assert min == a[2] == 1;
18 }
```

B.7.4 Exercise B2: Min-Heap Validation

Listing B.4: Binary min-heap validation with loop invariant

```

1  /* A binary min heap is a partially sorted binary tree where each node
2     has a value (or key) less or equal than the values of its descendent nodes.
3     It is conveniently stored in an array, with the root located at index 0,
4     and the parent of each non-root node 'i' located at index (i-1) mod 2.
5  */
6
7  // Check if an array 'a' stores a binary min heap.
8  method IsMinHeap(a: array<int>) returns (result: bool)
9  {
10     if a.Length > 0 {
11         for i := 1 to a.Length
12         {
13             if a[parent(i)] > a[i] {
14                 return false;
15             }
16         }
17     }
18     return true;
19 }
20
21 // Auxiliary function that gives the parent index of a non-root node 'i'.
22 function parent(i: nat): nat
23     requires i > 0
24     { (i-1)/2 }
25
26 // Test cases checked statically.
27 method TestIsMinHeap(){
28     // Positive case
29     var a1:= new int[] [2, 4, 3, 5, 10, 3];
30     var res1 := IsMinHeap(a1);
31     assert res1;
32
33     // Negative case
34     var a2 := new int[] [2, 1, 4];
35     var res2 := IsMinHeap(a2);
36     assert parent(1) == 0 && a2[0] > a2[1]; // proof helper (counter-example)
37     assert !res2;
38 }

```

B.7.5 Exercise C1: Selection Sort Algorithm

Listing B.5: Selection sort with comprehensive loop invariants

```

1  /*
2  * Formal verification with Dafny of the selection sort algorithm
3  * described in https://en.wikipedia.org/wiki/Selection\_sort
4  */
5
6  // Sorts array 'a' using the selection sort algorithm.
7  method SelectionSort(a: array<int>)
8      modifies a
9  {
10     // In each iteration, finds the minimum value in the unsorted right-hand side of the
        array
11     // and appends it (by swapping) to the sorted left-hand side of the array.
12     for i := 0 to a.Length
13     {
14         var minIndex := FindMinIndex(a, i, a.Length);
15         if minIndex != i {
16             a[i], a[minIndex] := a[minIndex], a[i]; // swap
17         }
18     }
19 }
20
21 // Finds the index of the minimum value in a non-empty subarray a[begin .. end], with begin <
    end.
22 method FindMinIndex(a: array<int>, begin: nat := 0, end: nat := a.Length) returns (minIndex:
    int)
23 {
24     minIndex := begin;
25     for i := begin + 1 to end
26     {
27         if a[i] < a[minIndex] {
28             minIndex := i;
29         }
30     }
31     return minIndex;
32 }
33
34 // Test case checked statically.
35 method TestSelectionSort() {
36     var a := new int[] [9, 4, 6, 1, 8];
37     assert a[..] == [9, 4, 6, 1, 8]; // proof helper
38     SelectionSort(a);
39     assert a[..] == [1, 4, 6, 8, 9];
40 }

```

B.7.6 Exercise C2: Array Reversal

Listing B.6: Generic array reversal with complex loop invariants

```
1 // Reverses the elements in array 'a'.
2 method Reverse<T>(a: array<T>)
3     modifies a
4 {
5     var i := 0; // left index
6     var j := a.Length - 1; // right index
7     while i < j
8     {
9         a[i], a[j] := a[j], a[i]; // swap
10        i := i + 1;
11        j := j - 1;
12    }
13 }
14
15 // Test case checked statically.
16 method TestReverse(){
17     var a1 := new int[] [1, 2, 3, 4];
18     Reverse(a1);
19     assert a1[..] == [4, 3, 2, 1];
20     Reverse(a1);
21     assert a1[..] == [1, 2, 3, 4];
22 }
```

This section presents the comprehensive feedback questionnaire administered to participants following completion of the experimental trial. The questionnaire is structured into four distinct sections designed to capture essential demographic information, detailed task performance metrics, systematic usability assessment through standardized System Usability Scale (SUS) questions, and qualitative feedback regarding participant experiences with the enhanced plugin.

Name *

Your answer

Previous experience with Dafny *

1

2

3

4

5

No experience

Expert level

Experimental group *

Group 1

Group 2

[illegible]

I felt very confident using the plugin *

	1	2	3	4	5	
Strongly disagree	☐	☐	☐	☐	☐	Strongly agree

I needed to learn a lot of things before I could get going with this plugin *

	1	2	3	4	5	
Strongly disagree	☐	☐	☐	☐	☐	Strongly agree

I found the plugin useful to help software engineers verifying programs in Dafny *

	1	2	3	4	5	
Strongly disagree	☐	☐	☐	☐	☐	Strongly agree

B.8.4 Section 4: Additional Feedback

What are the main benefits you found in using the plugin? *

Your answer

What were the main difficulties and limitations you encountered when using the plugin? *

Your answer

What improvements do you suggest?

Your answer